Container Runtimes & CNI

Kubernetes Production · Lesson 5

Provisioning the data plane with Kubespray

`kubespray.io` · `kubernetes-sigs/kubespray`

Learning Objectives

By the end of this lesson you will be able to:

- Select the container runtime with `container_manager` (default **containerd**)
- Configure **containerd** — `SystemdCgroup`, registry mirrors — and **CRI-O**
- Layer **sandboxed runtimes** (gVisor, Kata) on containerd via **RuntimeClass**
- Select the CNI with `kube_network_plugin` (default **calico**)
- Set the **pod / service CIDRs** and `kube_proxy_mode`
- Compare **Calico, Cilium and Flannel** as provisioning choices

> Reference: *Kubespray docs — CRI/ and CNI/* — kubespray.io

Provisioning vs the Network Model

This lesson is the **install-time** decision: *which* runtime and *which* CNI you wire into `group_vars` **before** deploying the cluster.

This lesson (Lesson 5)	Comes later (Lesson 13)
`container_manager` — pick the runtime	How the runtime runs your Pods
`kube_network_plugin` — pick the CNI	The pod-to-pod / service network model
Pod & service **CIDRs**, `kube_proxy_mode`	**NetworkPolicy** enforcement
Encapsulation / BGP backend	**DNS** (CoreDNS) resolution

> Calico and Cilium *can* enforce policy — but **NetworkPolicy and DNS** are a workload concern covered in **Lesson 13**. Here we only choose and wire them.

5.1 Container Runtimes (CRI)

`container_manager` — the CRI Switch

One variable decides which **Container Runtime Interface** the kubelet talks to.

```
# group_vars/k8s_cluster/k8s-cluster.yml
container_manager: containerd   # default; or: crio
```

Value	Runtime
`containerd`	**Default** — industry-standard, runs `runc` by default
`crio`	Lightweight CRI implementation (extra `all.yml` keys needed)

> **gVisor** and **Kata** are **not** values here — they are extra runtimes layered on **containerd** and chosen **per-Pod** via RuntimeClass.

containerd — Default + `SystemdCgroup`

containerd is the default; it runs containers with the `runc` OCI runtime, configured via the `containerd_runc_runtime` dictionary.

```
containerd_runc_runtime:
  name: runc
  type: "io.containerd.runc.v2"
  options:
    Root: ""
    SystemdCgroup: "false"
    BinaryName: /usr/local/bin/my-runc
  base_runtime_spec: cri-base.json
```

- `SystemdCgroup` controls systemd cgroup integration — it **must match the kubelet's cgroup driver**.
- Extra runtimes via `containerd_additional_runtimes`; `containerd_default_runtime` changes which one is default.

> With containerd, use `etcd_deployment_type: host`.

containerd — Registry Mirrors

Mirrors are a **list** under `containerd_registries_mirrors`. There is **no separate insecure key** — an HTTP registry is a mirror with `skip_verify: true`.

```
containerd_registries_mirrors:
  - prefix: docker.io
    mirrors:
      - host: https://mirror.gcr.io
        capabilities: ["pull", "resolve"]
        skip_verify: false
    server: https://registry-1.docker.io
  - prefix: 172.19.16.11:5000          # private / insecure registry
    mirrors:
      - host: http://172.19.16.11:5000
        capabilities: ["pull", "resolve"]
        skip_verify: true
```

> The old `containerd_registries` / `containerd_insecure_registries` variables are **deprecated** — use `containerd_registries_mirrors`.

CRI-O — Enabling & Registries

Set `container_manager: crio` plus a few required keys in `all/all.yml`:

```
container_manager: crio
download_container: false
skip_downloads: false
etcd_deployment_type: host   # or kubeadm
```

```
crio_registries:
  - prefix: docker.io
    location: registry-1.docker.io
    mirrors:
      - location: 192.168.100.100:5000
        insecure: true
```

- **`crio_insecure_registries`** — list of HTTP registries.
- **`crio_registry_auth`** — per-registry `registry` / `username` / `password`.
- **`crio_runtimes`** — add runtimes (e.g. user namespaces with `crio_remap_enable`).

gVisor & Kata — Layered on containerd

Both need "a container manager compatible with selecting the RuntimeClass such as **containerd**." Enable them *in addition* to containerd.

```
container_manager: containerd
gvisor_enabled: true            # gVisor — runsc application kernel
kata_containers_enabled: true   # Kata — lightweight micro-VMs (QEMU)
etcd_deployment_type: host

# Kata Pod Overhead (accounts for the VM's resources)
kubelet_cgroup_driver: cgroupfs           # or 'systemd' with cgroups v2
kata_containers_qemu_overhead: true
kata_containers_qemu_overhead_fixed_cpu: 10m
kata_containers_qemu_overhead_fixed_memory: 290Mi
```

- **gVisor** — Go application kernel; ships the `runsc` OCI runtime → extra isolation. RuntimeClass name `gvisor` (handler `runsc`).
- **Kata** — each Pod in its own micro-VM. RuntimeClass `kata-qemu` (auto-generated).

Per-Workload RuntimeClass

With sandboxed runtimes enabled, every Pod still uses `runc` unless it **opts in** by name with `runtimeClassName`.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  runtimeClassName: gvisor      # or: kata-qemu
  containers:
  - name: nginx
    image: nginx:1.14.2
```

`runtimeClassName`	Result
(omitted)	Default `runc` — standard container
`gvisor`	gVisor sandbox (handler `runsc`)
`kata-qemu`	Kata micro-VM isolation

5.2 Networking & CNI

`kube_network_plugin` — the CNI Switch

One variable picks the CNI. Kubespray installs and configures it for you.

```
# group_vars/k8s_cluster/k8s-cluster.yml
kube_network_plugin: calico   # default
```

Value	Plugin
`calico`	**Default** — mature, NetworkPolicy, BGP
`cilium`	eBPF datapath, kube-proxy replacement
`flannel`	Simple VXLAN overlay
`kube-ovn` · `kube-router`	OVN/OVS SDN · LVS/IPVS + BGP
`multus`	Meta-plugin — extra interfaces beside a primary CNI
`macvlan` · `cni` · `cloud`	L2 · generic (you supply config) · provider routing

> `cni` only unpacks plugin binaries (`cni_version`) into `/opt/cni/bin` — you must supply `/etc/cni/net.d` yourself.

Pod & Service CIDRs

Plugin-independent ranges that define the cluster network — pick them **before** deploying and ensure they **do not overlap** existing infra.

Variable	Default	Meaning
`kube_service_addresses`	`10.233.0.0/18`	**Service** ClusterIP range
`kube_pods_subnet`	`10.233.64.0/18`	**Pod** IP range
`kube_network_node_prefix`	`24`	Per-node pod subnet carved from the pod CIDR
`kube_proxy_mode`	`ipvs`	`ipvs`, `iptables`, or `nftables`

- Each node gets a `/24` slice of `kube_pods_subnet` for its Pods.

> Some plugins (Cilium / eBPF) **replace kube-proxy** entirely — the `kube_proxy_mode` plumbing is then bypassed.

5.3 Calico (the default CNI)

Calico — Datastore & Backend

The most mature, feature-rich option; `kube_network_plugin: calico`. Configured in the sample `k8s-net-calico.yml`.

```
calico_datastore: "kdd"             # Kubernetes datastore (default); or 'etcd'
calico_network_backend: vxlan       # default; 'bird' enables BGP; or 'none'
calico_vxlan_mode: Always           # default — VXLAN is the more mature impl
calico_ipip_mode: Never             # default; needs calico_network_backend: bird
```

- **kdd + > 50 nodes** → enable the **Typha** daemon to offload the API server.
- Encapsulation is set **independently per overlay** (`Always` / `CrossSubnet` / `Never`); VXLAN is on by default, IPIP requires the `bird` backend.

Calico — BGP & Route Reflectors

For routed, **encap-free** fabrics, peer Calico with your datacenter routers.

```
peer_with_router: true              # peer with ToR / border routers
global_as_num: "64512"              # cluster BGP AS number (per-node local_as)
nat_outgoing: true                  # SNAT pod traffic leaving the pod CIDR
calico_advertise_cluster_ips: true # advertise Service IPs over BGP
```

- **Route reflectors** — for large fabrics, dedicate nodes to the `calico_rr` inventory group (`cluster_id`, `calico_rr_id`, `calico_group_id`) so nodes peer with reflectors instead of a **full-mesh BGP**.

> Calico is also a full **policy engine** (NetworkPolicy + `GlobalNetworkPolicy`) — but *enforcing* policy is a **Lesson 13** topic.

5.4 Cilium, Flannel & Others

Cilium — eBPF Datapath

Modern, eBPF-based plugin; `kube_network_plugin: cilium`.

```
cilium_kube_proxy_replacement: true   # eliminate kube-proxy
cilium_enable_hubble: true            # observability
cilium_hubble_install: true           # relay + UI
cilium_enable_bgp_control_plane: true # modern BGP control plane
cilium_encryption_enabled: true
cilium_encryption_type: wireguard     # or 'ipsec'
```

- **kube-proxy replacement** — on Cilium < v1.16 use `strict`; Kubespray converts `strict` → `true`. Needs the kube-apiserver address for all components.
- **Hubble** gives flow-level observability (DNS, drops, HTTP) and a UI.

Flannel — Simple VXLAN

Minimal overlay; `kube_network_plugin: flannel`.

- Backends: **`vxlan`** (default), `host-gw`, `wireguard`.
- Writes `/run/flannel/subnet.env` (`FLANNEL_NETWORK`, `FLANNEL_SUBNET`, …).
- **No NetworkPolicy** — Flannel provides **pod connectivity only**; use Calico/Cilium if you need policy enforcement.

> Known issue: a VXLAN-backend bug needs the workaround `ethtool --offload flannel.1 rx off tx off` until a fixed version ships.

Kube-OVN · kube-router · Multus · generic

Plugin	What it offers
`kube-ovn`	OVN/OVS SDN — per-namespace subnets, VPCs, QoS
`kube-router`	Lightweight — LVS/IPVS services, iptables policy, BGP
`multus`	Meta-plugin — attach **extra** interfaces beside a primary CNI
`macvlan`	Direct L2 attachment
`cloud`	Let the cloud provider handle routing
`cni`	Generic — Kubespray drops binaries, **you** supply `/etc/cni/net.d`

> Multus is layered *on top of* a primary CNI (e.g. Calico) — useful for Pods that need a second NIC (storage, SR-IOV, telco data planes).

How to Pick a CNI

If you need…	Choose
Default, mature, NetworkPolicy, BGP to ToR, large scale	**Calico**
eBPF datapath, kube-proxy replacement, L7 policy, Hubble	**Cilium**
Simplest possible overlay, no policy needed	**Flannel**
OVN-style SDN (per-namespace subnets, VPC)	**Kube-OVN**
Multiple interfaces per Pod	**Multus** + a primary CNI
You supply your own CNI config	**cni** (generic)

> Default to **Calico** unless a specific need (eBPF, OVN, minimalism) points elsewhere. The choice is one line in `group_vars`; the **CIDRs** are the real decision.

Key Takeaways

- `container_manager` = the runtime switch: `containerd` (default) or `crio`
- containerd: `SystemdCgroup` must match the kubelet; mirrors via `containerd_registries_mirrors` (insecure = `skip_verify: true`)
- **gVisor (`gvisor` / `runsc`)** & **Kata (`kata-qemu`)** layer on containerd, chosen **per-Pod** via **RuntimeClass**
- `kube_network_plugin` = the CNI switch (default `calico`)
- CIDRs: services `10.233.0.0/18`, pods `10.233.64.0/18`, node prefix `24`; `kube_proxy_mode` = `ipvs` / `iptables` / `nftables`
- **Calico** = VXLAN + BGP/route-reflectors · **Cilium** = eBPF + Hubble · **Flannel** = simple overlay, **no policy**
- This is the **provisioning** choice — **NetworkPolicy & DNS** are **Lesson 13**

End of Lesson 5

Next: Lesson 6 — Deploying the Cluster

`container_manager: containerd` · `kube_network_plugin: calico` · `runtimeClassName: gvisor|kata-qemu` · CIDRs `10.233.0.0/18` / `10.233.64.0/18`

</content>